

ROYAUME DU MAROC

Ministère de l'Éducation Nationale, du Préscolaire et des Sports

Centre de Formation Al Kendi – Casablanca

Brevet de Technicien Supérieur (BTS)

Filière : Développement des Applications Informatiques



RAPPORT DE PROJET DE FIN D'ÉTUDES

PulmoScan AI

*Application mobile d'aide au diagnostic des pathologies
pulmonaires par intelligence artificielle embarquée*

sur radiographies thoraciques

Réalisé par :

Foudali Kenza

El Hajji Adnane

Encadré par :

Mme. Amina Aboulmira

Année universitaire : 2025 / 2026

Soutenu en Juin 2026

Dédicaces

*À nos chers parents,
qui n'ont jamais cessé de nous soutenir, de nous encourager
et de croire en nous tout au long de ce parcours.*

*À nos frères et sœurs,
pour leur présence, leur patience et leur affection.*

*À nos enseignants et formateurs,
qui ont semé en nous le goût du savoir et de l'effort.*

*À tous nos amis et camarades de promotion,
avec qui nous avons partagé les meilleurs moments
de notre formation.*

*À toutes les personnes qui, de près ou de loin,
ont contribué à la réussite de ce travail.*

Nous dédions ce modeste travail.

— *Kenza & Adnane*

Remerciements

Au terme de ce projet de fin d'études, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous adressons nos remerciements les plus sincères à notre encadrante, **Mme. Amina Aboulmira**, pour la qualité de son encadrement, sa disponibilité constante, ses conseils avisés et ses encouragements tout au long de ce projet. Son expertise et sa rigueur scientifique ont été déterminantes pour mener à bien ce travail.

Nous remercions également l'ensemble du corps enseignant et administratif du **Centre de Formation Al Kendi** de Casablanca, qui nous a accompagnés durant ces deux années de formation au sein de la filière BTS Développement des Applications Informatiques. La solidité des connaissances que nous avons acquises constitue le socle sur lequel repose ce projet.

Nos remerciements vont aussi aux membres du jury qui nous font l'honneur d'évaluer et de juger ce modeste travail. Nous espérons qu'il sera à la hauteur de leurs attentes.

Enfin, nous exprimons toute notre reconnaissance à nos familles et à nos proches pour leur soutien indéfectible, leur patience et leurs encouragements qui nous ont permis de surmonter les moments les plus difficiles.

À tous, nous disons sincèrement **merci**.

Résumé

Le présent projet de fin d'études porte sur la conception et la réalisation de **PulmoScan AI**, une application mobile multiplateforme d'aide au diagnostic des pathologies pulmonaires à partir de radiographies thoraciques. Dans un contexte marqué par une prévalence élevée des maladies respiratoires au Maroc et par une pénurie de radiologues, en particulier dans les zones périphériques, les délais de diagnostic peuvent atteindre plusieurs jours. PulmoScan AI propose une réponse à cette problématique en mettant à disposition du personnel médical un outil d'assistance fonctionnant entièrement **hors connexion** (*offline-first*).

L'application, développée avec le framework **Flutter 3.22** et le langage **Dart 3.4**, embarque deux modèles d'intelligence artificielle exécutés localement grâce à **TensorFlow Lite**. Le premier, un réseau **DenseNet121** issu de la bibliothèque TorchXRayVision et entraîné sur le jeu de données NIH ChestX-ray14 (112 120 images), réalise la classification de 14 pathologies thoraciques. Le second, un réseau **U-Net**, assure la segmentation des champs pulmonaires. Les données patients et les examens sont stockés localement dans une base **SQLite**, garantissant la confidentialité et l'indépendance vis-à-vis d'un serveur distant.

Ce rapport décrit l'ensemble du cycle de développement, depuis l'analyse du contexte et l'expression des besoins jusqu'à la conception architecturale (UML) et la réalisation technique, en passant par les difficultés rencontrées et les solutions apportées.

Mots-clés : Intelligence artificielle, apprentissage profond, radiographie thoracique, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, aide au diagnostic, santé mobile.

Abstract

This graduation project deals with the design and implementation of **PulmoScan AI**, a cross-platform mobile application that assists in the diagnosis of pulmonary pathologies from chest X-rays. In a context characterized by a high prevalence of respiratory diseases in Morocco and a shortage of radiologists, especially in peripheral facilities, diagnostic delays can reach several days. PulmoScan AI addresses this issue by providing medical staff with an assistance tool that runs entirely **offline** (*offline-first*).

The application, built with the **Flutter 3.22** framework and the **Dart 3.4** language, embeds two artificial intelligence models that run locally thanks to **TensorFlow Lite**. The first one, a **DenseNet121** network from the TorchXRayVision library, trained on the NIH ChestX-ray14 dataset (112,120 images), performs the classification of 14 thoracic pathologies. The second one, a **U-Net** network, handles the segmentation of lung fields. Patient data and examinations are stored locally in an **SQLite** database, ensuring confidentiality and independence from any remote server.

This report describes the entire development cycle, from context analysis and requirements specification to architectural design (UML) and technical implementation, including the difficulties encountered and the solutions adopted.

Keywords : Artificial intelligence, deep learning, chest X-ray, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, computer-aided diagnosis, mobile health.

Table des matières

Dédicaces	i
Remerciements	iii
Résumé	iv
Abstract	v
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xi
Introduction générale	1
1 Contexte et Problématique	3
Introduction	3
1.1 Contexte général	3
1.1.1 Les maladies respiratoires au Maroc	3
1.1.2 La place de l'intelligence artificielle en imagerie médicale	4
1.2 Problématique	4
1.3 Étude de l'existant	5
1.3.1 Solutions cloud	5
1.3.2 Solutions logicielles de poste fixe	5
1.3.3 Positionnement de PulmoScan AI	5
1.4 Objectifs du projet	6
1.5 Méthodologie et planification	6
Conclusion	6
2 Analyse des Besoins	7
Introduction	7

2.1	Identification des acteurs	7
2.2	Besoins fonctionnels	7
2.3	Besoins non fonctionnels	8
2.4	Diagramme de cas d'utilisation	8
2.4.1	Description textuelle d'un cas d'utilisation	9
2.5	Product Backlog	9
	Conclusion	9
3	Conception	11
	Introduction	11
3.1	Architecture générale	11
3.2	Modélisation UML	12
3.2.1	Diagramme de classes	12
3.2.2	Diagrammes de séquence	13
3.3	Conception de la base de données	13
3.4	Conception des modèles d'intelligence artificielle	15
3.4.1	DenseNet121 – Classification	15
3.4.2	U-Net – Segmentation	15
	Conclusion	16
4	Réalisation	17
	Introduction	17
4.1	Environnement et outils de développement	17
4.2	Pile technologique	17
4.3	Organisation du code source	17
4.4	Le Design System V4	18
4.5	Présentation des écrans	19
4.6	Implémentation de l'inférence IA	20
4.6.1	Prétraitement de l'image	20
4.6.2	Post-traitement et décision	21
4.7	Données de démonstration	22
4.8	Difficultés rencontrées et solutions	22
4.9	Répartition du travail	22
	Conclusion	22
	Conclusion générale	24
	Bibliographie	26
A	Annexes	27
A.1	Annexe A – Liste complète des 14 pathologies	27

A.2	Annexe B – Extrait du chargement des modèles	28
A.3	Annexe C – Maquettes Figma	28

Liste des figures

Liste des tableaux

1.1	Comparaison de PulmoScan AI avec les solutions existantes	5
2.1	Besoins non fonctionnels du système	8
2.2	Product Backlog – fonctionnalités implémentées	10
3.1	Schéma de la base de données SQLite (version 4)	14
3.2	Caractéristiques du modèle DenseNet121	15
3.3	Caractéristiques du modèle U-Net	16
4.1	Outils et environnement de développement	17
4.2	Pile technologique de l’application	18
4.3	Palette de couleurs du Design System V4	19
4.4	Seuils et paramètres de décision de l’IA	22
4.5	Patients de démonstration	22
4.6	Difficultés rencontrées et solutions apportées	23
4.7	Répartition du travail entre les membres du binôme	23
A.1	Les 14 pathologies classifiées et leur fiabilité	27

Liste des abréviations

Abréviation	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
AUC	Area Under the Curve (aire sous la courbe ROC)
BTS	Brevet de Technicien Supérieur
CNN	Convolutional Neural Network (réseau de neurones convolutif)
CRUD	Create, Read, Update, Delete
DICOM	Digital Imaging and Communications in Medicine
IDE	Integrated Development Environment
ML	Machine Learning (apprentissage automatique)
NIH	National Institutes of Health
PFE	Projet de Fin d'Études
ROC	Receiver Operating Characteristic
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
TFLite	TensorFlow Lite
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

Introduction générale

L'intelligence artificielle (IA) connaît aujourd'hui un essor considérable dans le domaine de la santé. Parmi ses applications les plus prometteuses figure l'analyse automatisée des images médicales, qui ouvre la voie à des outils d'aide au diagnostic capables d'assister les professionnels de santé dans leurs décisions cliniques. L'imagerie thoracique, et plus particulièrement la radiographie pulmonaire, constitue un terrain privilégié pour ces technologies en raison de la grande disponibilité des appareils de radiographie et du volume important d'examens réalisés quotidiennement.

Au Maroc, les maladies respiratoires représentent une charge sanitaire majeure. Pourtant, le nombre de radiologues qualifiés demeure insuffisant, et leur répartition géographique est très inégale : les grandes villes concentrent l'essentiel des spécialistes, tandis que les zones rurales et périphériques en sont largement dépourvues. Cette situation engendre des délais de diagnostic qui peuvent atteindre plusieurs jours, au détriment de la prise en charge des patients. Par ailleurs, la connectivité internet limitée dans ces régions constitue un obstacle supplémentaire à l'adoption de solutions reposant sur le *cloud*.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : la conception et la réalisation de **PulmoScan AI**, une application mobile d'aide au diagnostic des pathologies pulmonaires. L'originalité de notre approche réside dans son caractère *offline-first* : l'ensemble des traitements, y compris l'inférence des modèles d'intelligence artificielle, s'effectue directement sur l'appareil mobile, sans nécessiter de connexion à un serveur distant. Cette architecture garantit à la fois la confidentialité des données patients et l'utilisabilité de l'outil dans les environnements à faible connectivité.

L'application repose sur deux modèles de réseaux de neurones profonds. Un modèle **DenseNet121**, entraîné sur le jeu de données de référence NIH ChestX-ray14, classe 14 pathologies thoraciques. Un modèle **U-Net** segmente les champs pulmonaires afin de produire une visualisation explicative. Ces modèles, convertis au format TensorFlow Lite, sont embarqués dans une application développée avec Flutter, ce qui assure une compatibilité multiplateforme entre les systèmes Android et iOS.

Le présent rapport rend compte de la démarche suivie pour mener à bien ce projet. Il s'articule autour de quatre chapitres. Le **premier chapitre** présente le contexte général, la problématique et les objectifs du projet. Le **deuxième chapitre** est consacré à l'analyse des besoins fonctionnels et non fonctionnels, ainsi qu'à la spécification des acteurs et des cas d'utilisation. Le **troisième chapitre** détaille la conception du système, à travers la modélisation UML, l'architecture logicielle et le schéma de la base de données. Enfin, le **quatrième chapitre** décrit la réalisation technique, les outils employés, les principaux écrans de l'application et les difficultés rencontrées. Une conclusion générale dresse le bilan du travail accompli et ouvre des perspectives d'évolution.

CHAPITRE 1

Contexte et Problématique

Introduction

Ce premier chapitre pose les fondements du projet. Il décrit le contexte sanitaire et technologique dans lequel s'inscrit PulmoScan AI, formule la problématique à laquelle l'application entend répondre, présente une étude des solutions existantes, puis énonce les objectifs et le cadre méthodologique du projet.

1.1 Contexte général

1.1.1 Les maladies respiratoires au Maroc

Les affections de l'appareil respiratoire constituent l'une des principales causes de consultation et d'hospitalisation au Maroc. Pneumonies, épanchements pleuraux, œdèmes pulmonaires, emphysèmes ou encore tuberculose figurent parmi les pathologies fréquemment rencontrées. Le diagnostic de ces affections repose en grande partie sur l'examen radiographique du thorax, qui demeure l'examen d'imagerie de première intention en raison de son faible coût, de sa rapidité d'exécution et de sa large disponibilité.

Toutefois, l'interprétation d'une radiographie thoracique est un acte médical complexe qui requiert une expertise spécialisée. Or, le système de santé marocain souffre d'un déficit chronique de radiologues. Ce déficit est particulièrement marqué dans les provinces éloignées des grands centres urbains, où les structures de soins de proximité ne disposent pas toujours d'un radiologue résident.

1.1.2 La place de l'intelligence artificielle en imagerie médicale

L'apprentissage profond (*deep learning*) a révolutionné le domaine de la vision par ordinateur au cours de la dernière décennie. Les réseaux de neurones convolutifs (CNN) ont démontré des performances remarquables dans les tâches de classification et de détection d'images, atteignant parfois des niveaux comparables à ceux d'experts humains pour certaines pathologies.

En imagerie thoracique, la mise à disposition publique de grands jeux de données annotés, tels que NIH ChestX-ray14, a stimulé le développement de modèles capables de détecter automatiquement plusieurs pathologies. Ces modèles ne se substituent pas au médecin, mais constituent une aide à la décision susceptible de réduire les délais, de prioriser les cas urgents et d'apporter un second avis dans les contextes de pénurie d'expertise.

Le jeu de données NIH ChestX-ray14

Publié par le National Institutes of Health (NIH) en 2017, le jeu de données ChestX-ray14 regroupe **112 120 radiographies thoraciques** provenant de 30 805 patients, annotées pour **14 pathologies** distinctes. Il constitue aujourd'hui une référence pour l'entraînement et l'évaluation des modèles de classification d'images thoraciques.

1.2 Problématique

La problématique centrale de ce projet peut être formulée ainsi :

Problématique

Comment concevoir un outil d'aide au diagnostic des pathologies pulmonaires qui soit **accessible, rapide, respectueux de la confidentialité** des données patients et **utilisable dans les zones à faible connectivité**, afin de réduire les délais de diagnostic et de pallier la pénurie de radiologues ?

De cette problématique découlent plusieurs sous-questions :

- Comment exécuter des modèles d'intelligence artificielle directement sur un appareil mobile, sans serveur distant ?
- Comment garantir la confidentialité des données médicales sensibles ?
- Comment offrir une interface ergonomique et intuitive au personnel médical, y compris non spécialiste de l'informatique ?
- Comment assurer la fiabilité des résultats produits tout en signalant clairement leurs limites ?

1.3 Étude de l'existant

Plusieurs solutions d'aide au diagnostic par imagerie existent sur le marché ou dans la littérature scientifique. On peut les regrouper en trois catégories.

1.3.1 Solutions cloud

La majorité des plateformes commerciales reposent sur une architecture *cloud* : l'image est transmise à un serveur distant qui exécute l'inférence et renvoie le résultat. Si cette approche offre une grande puissance de calcul, elle présente des inconvénients majeurs dans notre contexte : dépendance à une connexion internet stable, latence réseau, et surtout transfert de données médicales sensibles vers des serveurs tiers, ce qui pose des problèmes de confidentialité.

1.3.2 Solutions logicielles de poste fixe

D'autres solutions s'intègrent aux stations de travail des services de radiologie (PACS). Elles sont performantes mais coûteuses, lourdes à déployer et inadaptées aux structures de soins de proximité dépourvues d'infrastructure dédiée.

1.3.3 Positionnement de PulmoScan AI

Le tableau suivant synthétise le positionnement de notre solution par rapport aux approches existantes.

TABLE 1.1 – Comparaison de PulmoScan AI avec les solutions existantes

Critère	Solution cloud	Poste fixe	PulmoScan AI
Fonctionnement hors ligne	Non	Oui	Oui
Confidentialité (données locales)	Faible	Élevée	Élevée
Mobilité	Élevée	Faible	Élevée
Coût de déploiement	Variable	Élevé	Faible
Connexion requise	Oui	Non	Non

PulmoScan AI se distingue par son architecture *offline-first* qui combine les avantages de la mobilité et de la confidentialité, tout en restant accessible et peu coûteuse.

1.4 Objectifs du projet

Le projet poursuit les objectifs suivants :

- Concevoir une application mobile multiplateforme (Android / iOS) d'aide au diagnostic des pathologies pulmonaires ;
- Embarquer des modèles d'IA s'exécutant localement sur l'appareil, sans serveur distant ;
- Classifier automatiquement 14 pathologies thoraciques à partir d'une radiographie ;
- Fournir une segmentation des champs pulmonaires comme support explicatif ;
- Assurer la gestion des patients et des examens dans une base de données locale ;
- Proposer une interface ergonomique adaptée au personnel médical.

1.5 Méthodologie et planification

Le développement a été conduit selon une démarche **itérative et incrémentale** inspirée des méthodes agiles. Le travail a été organisé en phases successives : étude et cadrage, analyse des besoins, conception, préparation et conversion des modèles d'IA, développement de l'application, intégration, puis tests et validation. Un diagramme de Gantt a permis de planifier et de suivre l'avancement.

[FIGURE]

Diagramme de Gantt présentant la planification temporelle du projet (phases d'analyse, conception, développement des modèles, développement applicatif, intégration et tests).

Conclusion

Ce chapitre a mis en évidence le besoin d'un outil d'aide au diagnostic pulmonaire adapté au contexte marocain. La problématique de l'accessibilité, de la confidentialité et de la connectivité justifie le choix d'une architecture *offline-first*. Le chapitre suivant approfondit l'analyse des besoins du système.

CHAPITRE 2

Analyse des Besoins

Introduction

Ce chapitre identifie les acteurs du système, recense les besoins fonctionnels et non fonctionnels, modélise les cas d'utilisation et organise les fonctionnalités sous forme d'un *product backlog*.

2.1 Identification des acteurs

Trois acteurs principaux interagissent avec le système :

Le Médecin

Acteur central, il lance les analyses, consulte les résultats produits par l'IA, génère les rapports et consulte l'historique des examens.

L'Assistant Médical

Il assure la gestion administrative des dossiers : il crée et modifie les patients, capture les images radiographiques et consulte les dossiers patients.

L'Administrateur

Il gère le système et les comptes utilisateurs.

2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit rendre :

- **Authentification** : connexion et déconnexion sécurisées, option « se souvenir de moi ».

- **Gestion des patients** : création, consultation, modification, recherche et filtrage des dossiers patients.
- **Gestion des examens** : création d'un examen par import d'une image (galerie ou appareil photo).
- **Analyse par IA** : exécution des modèles DenseNet121 (classification) et U-Net (segmentation).
- **Affichage des résultats** : diagnostic principal, scores de confiance des 14 pathologies, badge de sévérité et superposition du masque de segmentation.
- **Tableau de bord** : affichage de statistiques et des examens récents.
- **Profil** : consultation du profil du médecin et des paramètres.

2.3 Besoins non fonctionnels

TABLE 2.1 – Besoins non fonctionnels du système

Critère	Description
Performance	Inférence des modèles en quelques secondes sur un appareil mobile standard.
Disponibilité	Fonctionnement intégral hors connexion (<i>offline-first</i>).
Confidentialité	Stockage local des données médicales, aucun transfert vers un serveur tiers.
Portabilité	Compatibilité multiplateforme Android et iOS via Flutter.
Ergonomie	Interface intuitive, claire et adaptée au personnel médical.
Fiabilité	Affichage transparent des scores de confiance et des limites des modèles.
Maintenabilité	Architecture modulaire, code organisé en services réutilisables.

2.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous synthétise les interactions entre les acteurs et le système.

[FIGURE]

Diagramme de cas d'utilisation global. Acteurs : Médecin, Assistant Médical, Administrateur. Cas d'utilisation du Médecin : lancerAnalyse, consulterResultat, genererRapport, consulterHistorique. Cas d'utilisation de l'Assistant : creerPatient, modifierPatient, capturerImage, consulterPatient.

2.4.1 Description textuelle d'un cas d'utilisation

Cas d'utilisation : Lancer une analyse IA

Acteur principal : Médecin

Précondition : L'utilisateur est authentifié et un patient est sélectionné.

Scénario nominal :

1. Le médecin sélectionne un patient.
2. Il importe une radiographie depuis la galerie ou l'appareil photo.
3. Le système prétraite l'image (recadrage, redimensionnement, normalisation).
4. Les modèles DenseNet121 et U-Net sont exécutés localement.
5. Le système affiche le diagnostic principal, les scores de confiance et le masque de segmentation.

Postcondition : L'examen et ses résultats sont enregistrés dans la base locale.

2.5 Product Backlog

L'ensemble des fonctionnalités a été organisé sous forme d'un *product backlog* regroupant les fonctionnalités effectivement implémentées.

Conclusion

Ce chapitre a permis de clarifier les attentes vis-à-vis du système en identifiant ses acteurs, ses besoins fonctionnels et non fonctionnels, et en structurant les fonctionnalités

TABLE 2.2 – Product Backlog – fonctionnalités implémentées

ID	Fonctionnalité	Description
F1	Authentification	Connexion, déconnexion, « se souvenir de moi ».
F2	Gestion des patients	CRUD, recherche et filtrage.
F3	Création d'examen	Import d'image (galerie / appareil photo).
F4	Analyse par IA	DenseNet121 (classification) + U-Net (segmentation).
F5	Affichage des résultats	Scores de confiance et masque de segmentation.
F6	Tableau de bord	Statistiques et examens récents.
F7	Profil médecin	Consultation du profil et des paramètres.

sous forme d'un backlog. Le chapitre suivant traduit ces besoins en une conception détaillée.

CHAPITRE 3

Conception

Introduction

Ce chapitre présente la conception du système. Il décrit l'architecture logicielle, la modélisation UML (diagramme de classes et diagrammes de séquence), le schéma de la base de données, ainsi que la conception des modèles d'intelligence artificielle.

3.1 Architecture générale

L'application adopte une architecture **offline-first** en couches. La couche de présentation (interfaces Flutter) communique avec une couche de services qui encapsule la logique métier. Cette couche de services s'appuie sur deux composants singletons : le service d'IA (**AIService**), responsable de l'inférence TensorFlow Lite, et le service de base de données (**DatabaseService**), responsable des opérations CRUD sur la base SQLite locale.

[FIGURE]

Diagramme d'architecture en couches : couche Présentation (8 écrans Flutter) – couche Services (AIService, DatabaseService) – couche Données (modèles TFLite, base SQLite). Aucun serveur distant.

Le patron Singleton

Les services `AIService` et `DatabaseService` sont implémentés selon le patron de conception **Singleton**. Cela garantit qu'une seule instance de chaque service existe pendant toute la durée de vie de l'application, évitant ainsi le rechargement coûteux des modèles d'IA et l'ouverture multiple de connexions à la base de données.

3.2 Modélisation UML

3.2.1 Diagramme de classes

Le diagramme de classes décrit la structure statique du domaine. La classe abstraite `Utilisateur` est spécialisée en `AssistantMedical`, `Médecin` et `Administrateur`. Les classes `Patient`, `Examen` et `Resultat` modélisent le cœur métier.

[FIGURE]

Diagramme de classes UML. Classe parente Utilisateur (id, name, email, password, role) héritée par AssistantMedical, Médecin et Administrateur. Classes Patient (1) – (*) Examen (1) – (1) Resultat. Associations et cardinalités.

3.2.2 Diagrammes de séquence

Plusieurs diagrammes de séquence ont été élaborés pour décrire la dynamique des principales interactions : *Authentication*, *Création de patient*, *Acquisition d'image* et *Analyse IA*.

[FIGURE]

Diagramme de séquence « Analyse IA » : Médecin → NewExamScreen → AIService (analyzeImage) → prétraitement → DenseNet121 → U-Net → post-traitement → ResultsScreen → DatabaseService (insertExam).

3.3 Conception de la base de données

La persistance repose sur une base **SQLite** locale (version 4 du schéma), gérée via le paquet `sqlite`. Elle comporte trois tables principales.

Le listing suivant illustre la création de la table `exams`.

TABLE 3.1 – Schéma de la base de données SQLite (version 4)

Table	Champ	Description
users	id	Identifiant (clé primaire)
	name	Nom de l'utilisateur
	email	Adresse e-mail
	password	Mot de passe
	role	Rôle (médecin / assistant / admin)
	id	Identifiant (clé primaire)
	firstName	Prénom
	lastName	Nom
	dateOfBirth	Date de naissance
	gender	Genre
	contact	Coordonnées
	createdAt	Date de création
exams	id	Identifiant (clé primaire)
	patientId	Clé étrangère vers patients
	imagePath	Chemin de l'image
	diagnostic	Diagnostic principal
	confidence	Confiance (0.0 – 1.0)
	severite	Sévérité (urgent / moyen / normal / modere / severe / faible)
	details	Scores des 14 pathologies (JSON)
	createdAt	Date de création

```

1 CREATE TABLE exams (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   patientId INTEGER NOT NULL,
4   imagePath TEXT,
5   diagnostic TEXT,
6   confidence REAL,
7   severite TEXT,
8   details TEXT,
9   createdAt TEXT,
10  FOREIGN KEY (patientId) REFERENCES patients(id)
11 );

```

Listing 3.1 – Création de la table exams (SQLite)

3.4 Conception des modèles d'intelligence artificielle

Deux modèles complémentaires constituent le cœur intelligent de l'application.

3.4.1 DenseNet121 – Classification

Le modèle de classification est un réseau **DenseNet121** provenant de la bibliothèque **TorchXRayVision** et entraîné sur NIH ChestX-ray14. Sa particularité réside dans la connectivité dense entre couches, qui favorise la réutilisation des caractéristiques et limite la disparition du gradient.

TABLE 3.2 – Caractéristiques du modèle DenseNet121

Paramètre	Valeur
Fichier	<code>pulmoscan_model.tflite</code> (13,4 Mo)
Entrée	[1, 224, 224, 1] float32, niveaux de gris
Sortie	[1, 18] float32, activation sigmoïde
Pathologies utiles	14 premières (indices 0–13)
Indices non entraînés	14–17 (valeur $\approx 0,5$)
AUC	0,80 – 0,94 (classes fiables)
Dataset	NIH ChestX-ray14 (112 120 images)

Ordre des étiquettes TorchXRayVision

Les sorties du modèle suivent l'ordre des `default_pathologies` de TorchXRayVision : *Atelectasis*, *Consolidation*, *Infiltration*, *Pneumothorax*, *Edema*, *Emphysema*, *Fibrosis*, *Effusion*, *Pneumonia*, *Pleural_Thickening*, *Cardiomegaly*, *Nodule*, *Mass*, *Hernia*. Cet ordre n'est pas alphabétique et diffère de celui du jeu NIH brut.

3.4.2 U-Net – Segmentation

Le second modèle est un réseau **U-Net**, architecture en forme de « U » composée d'un chemin de contraction (encodeur) et d'un chemin d'expansion (décodeur) reliés par des connexions de saut. Il produit un masque de probabilité segmentant les champs pulmonaires.

TABLE 3.3 – Caractéristiques du modèle U-Net

Paramètre	Valeur
Fichier	unet_lung.tflite (29,6 Mo)
Entrée	[1, 256, 256, 1] float32, niveaux de gris /255
Sortie	[1, 256, 256, 1] masque de probabilité
Seuil de binarisation	0,5
Quantification	Aucune (float32)

[**FIGURE**]

Schéma de l'architecture U-Net : encodeur (contraction) à gauche, décodeur (expansion) à droite, connexions de saut horizontales, sortie : masque de segmentation pulmonaire.

Conclusion

Ce chapitre a détaillé la conception architecturale et logicielle de PulmoScan AI, depuis l'organisation en couches jusqu'à la modélisation UML, le schéma de base de données et la conception des deux modèles d'IA. Le chapitre suivant aborde la réalisation concrète du système.

CHAPITRE 4

Réalisation

Introduction

Ce dernier chapitre présente la mise en œuvre concrète du projet : l’environnement et les outils de développement, la pile technologique, l’organisation du code, les principaux écrans de l’application, la logique d’inférence de l’IA et les difficultés rencontrées avec leurs solutions.

4.1 Environnement et outils de développement

TABLE 4.1 – Outils et environnement de développement

Outil	Usage
GitHub	Gestion de versions et collaboration
Figma	Conception des maquettes UI/UX
VS Code / Android Studio	Environnements de développement (IDE)
Google Colab	Entraînement et conversion des modèles
Python + TensorFlow	Préparation et conversion TFLite des modèles

4.2 Pile technologique

4.3 Organisation du code source

Le code est organisé de manière modulaire au sein du répertoire `lib/`, séparant clairement les écrans, les services, les widgets et le thème.

TABLE 4.2 – Pile technologique de l'application

Couche	Technologie	Version / Paquet
Frontend	Flutter / Dart	3.22 / 3.4
Base de données	SQLite	sqflite ^2.3.0
Inférence IA	TensorFlow Lite	tflite_flutter ^0.10.4
Traitement d'image	image	image ^4.1
Sélection d'image	image_picker	image_picker ^1.0
Préférences	shared_preferences	shared_preferences ^2.2

```

1 lib/
2 +-- main.dart
3 +-- theme/
4 |   +-- v4_theme.dart
5 +-- screens/                (8 ecrans)
6 |   +-- landing_screen.dart
7 |   +-- onboarding_screen.dart
8 |   +-- dashboard.dart
9 |   +-- patients_screen.dart
10 |   +-- patient_detail_screen.dart
11 |   +-- new_exam_screen.dart
12 |   +-- results_screen.dart
13 |   +-- profile_screen.dart
14 +-- services/
15 |   +-- ai_service.dart      (singleton, inference TFLite)
16 |   +-- database_service.dart (singleton, CRUD SQLite)
17 +-- widgets/
18     +-- aperture_mark.dart

```

Listing 4.1 – Arborescence du code source

4.4 Le Design System V4

Une identité visuelle cohérente a été conçue, baptisée **Design System V4**. Elle repose sur un thème sombre et un accent turquoise distinctif.

Un widget personnalisé, **ApertureMark**, anime un diaphragme (iris) et constitue la signature visuelle de l'application.

TABLE 4.3 – Palette de couleurs du Design System V4

Jeton	Code	Usage
V4.bg	#0A0F1A	Fond sombre
V4.teal	#34E5C5	Accent principal
V4.surface1	#131B2E	Cartes
V4.ink	#E8EDF6	Texte
V4.urgent	#FF5A5F	Sévérité urgente
V4.moyen	#FFB547	Sévérité moyenne
V4.normal	#34E5C5	Sévérité normale

4.5 Présentation des écrans

L'application compte huit écrans principaux.

1. **LandingScreen** – page d'accueil avec section héros défilante et formulaire de connexion, avec défilement animé vers le formulaire.
2. **OnboardingScreen** – trois diapositives (Capture / IA / Rapport), widget **ApertureMark**, **PageView**, état mémorisé via **SharedPreferences**.
3. **Dashboard** – cartes de statistiques, examens récents et cartes de modèles d'IA cliquables ouvrant une feuille de détails.
4. **PatientsScreen** – liste recherchable avec badges de statut (urgent / normal / à surveiller).
5. **PatientDetailScreen** – dossier du patient et historique des examens.
6. **NewExamScreen** – assistant par étapes : sélection du patient → import de l'image → exécution de l'IA.
7. **ResultsScreen** – diagnostic principal, badge de sévérité, histogramme de 14 barres de confiance (seuil d'alerte 0,15) et superposition du masque U-Net.
8. **ProfileScreen** – profil du médecin et paramètres.

[FIGURE]

Captures d'écran de l'application : LandingScreen, Onboarding, Dashboard et NewExamScreen présentées côte à côte.

[FIGURE]

Capture d'écran du ResultsScreen : diagnostic principal, badge de sévérité, histogramme des 14 scores de confiance et masque de segmentation U-Net superposé.

4.6 Implémentation de l'inférence IA

Le service `AIService` encapsule toute la logique d'inférence. Il est implémenté en singleton et charge les modèles depuis les *assets*, avec un mécanisme de repli (`_simulate()`) en cas d'absence des fichiers.

4.6.1 Prétraitement de l'image

Avant l'inférence, l'image est recadrée au centre en carré, redimensionnée en 224×224 , convertie en luminance selon la recommandation Rec.601, puis normalisée par division par 255.


```

1 // Recadrage centre carre puis redimensionnement 224x224
2 final cropped = _centerCropSquare(decoded);
3 final resized = copyResize(cropped, width: 224, height: 224);
4
5 // Conversion en luminance (Rec.601) et normalisation /255
6 final input = List.generate(1, (_) => List.generate(224, (y) =>
7   List.generate(224, (x) {
8     final p = resized.getPixel(x, y);
9     final lum = 0.299 * p.r + 0.587 * p.g + 0.114 * p.b;
10    return [lum / 255.0];
11  })));

```

Listing 4.2 – Prétraitement de l'image (Dart, simplifié)

4.6.2 Post-traitement et décision

Le post-traitement applique plusieurs règles métier pour produire un diagnostic fiable : exclusion des quatre classes ambiguës à faible AUC du choix du diagnostic principal, repli si le score brut maximal est trop faible, classement « Normal » sous un seuil bas, calcul d'une confiance bornée et attribution d'un niveau de sévérité.

```

1 // Classes ambiguës exclues du diagnostic principal (AUC < 0.80)
2 const ambiguous = {'Infiltration', 'Pneumonia', 'Nodule',
3   'Consolidation'};
4
5 // Selection de la meilleure pathologie hors classes ambiguës
6 final best = _argMaxExcluding(scores, ambiguous);
7 final bestRaw = scores[best];
8
9 String diagnostic;
10 if (bestRaw < 0.03) {
11   diagnostic = 'Normal';
12 } else if (bestRaw < 0.04) {
13   diagnostic = _fallbackLabel();
14 } else {
15   diagnostic = labels[best];
16 }
17
18 // Confiance bornée et sévérité
19 final confidence = (bestRaw * 2.5).clamp(0.55, 0.97);
20 final severite = bestRaw >= 0.20 ? 'urgent' : 'moyen';

```

Listing 4.3 – Post-traitement des prédictions (Dart, simplifié)

TABLE 4.4 – Seuils et paramètres de décision de l’IA

Paramètre	Valeur
Seuil « Normal »	score brut < 0,03
Seuil de repli	score brut < 0,04
Seuils par classe	0,04 – 0,10
Formule de confiance	(bestRaw × 2,5) borné à [0,55 ; 0,97]
Seuil de sévérité « urgent »	score brut ≥ 0,20
Seuil d’alerte (histogramme)	0,15
Seuil de binarisation U-Net	0,50

4.7 Données de démonstration

Afin de faciliter la démonstration et les tests, des données fictives sont préchargées dans la base.

TABLE 4.5 – Patients de démonstration

Patient	Diagnostic (confiance)
Ahmed Benali	Emphysème (87%)
Fatima Zahra	Pneumonie (92%)
Youssef Alaoui	Épanchement (78%)

Compte de démonstration

Identifiant : demo@pulmoscan.ma | Mot de passe : Demo1234

4.8 Difficultés rencontrées et solutions

Le développement a soulevé plusieurs difficultés techniques que nous avons su résoudre.

4.9 Répartition du travail

Le projet a été mené en binôme avec une répartition équilibrée des tâches.

Conclusion

Ce chapitre a présenté la réalisation effective de PulmoScan AI : la pile technologique, l’organisation du code, le design, les écrans, la logique d’inférence et la résolution des

TABLE 4.6 – Difficultés rencontrées et solutions apportées

Difficulté	Solution
Ordre des étiquettes de TorchXRayVision non alphabétique, entraînant un étiquetage erroné de toutes les prédictions.	Réalignement strict sur l'ordre <code>default_pathologies</code> de TorchX-RayVision.
Classes à faible AUC dominant le diagnostic principal.	Exclusion des 4 classes ambiguës (Infiltration, Pneumonia, Nodule, Consolidation) de la sélection.
Débordement de la barre de gestes Android sur l'onboarding.	Prise en compte de <code>MediaQuery.padding.bottom</code> .
Incohérence des jetons de sévérité en base.	Normalisation vers <code>modere</code> / <code>severe</code> / <code>faible</code> / <code>urgent</code> / <code>moyen</code> / <code>normal</code> .

TABLE 4.7 – Répartition du travail entre les membres du binôme

Membre	Contributions
Foudali Kenza	Modèles d'IA (notebooks, conversion TFLite), base de données SQLite, interface Flutter.
El Hajji Adnane	Interface Flutter (UI/UX), architecture logicielle, intégration.

difficultés techniques. L'application livrée répond aux besoins exprimés et démontre la faisabilité d'une solution d'IA médicale embarquée et hors ligne.

Conclusion générale

Ce projet de fin d'études nous a permis de concevoir et de réaliser **PulmoScan AI**, une application mobile d'aide au diagnostic des pathologies pulmonaires fonctionnant entièrement hors connexion. Partant d'un constat sanitaire concret – la pénurie de radiologues et les délais de diagnostic au Maroc, notamment dans les zones à faible connectivité – nous avons proposé une solution *offline-first* qui embarque deux modèles d'intelligence artificielle directement sur l'appareil de l'utilisateur.

Sur le plan technique, nous avons intégré un modèle de classification **DenseNet121** capable de reconnaître 14 pathologies thoraciques et un modèle de segmentation **U-Net** produisant un masque explicatif des champs pulmonaires. Ces modèles, convertis au format TensorFlow Lite, sont exécutés localement au sein d'une application Flutter multiplateforme. La persistance des données est assurée par une base SQLite locale, garantissant la confidentialité des informations médicales.

Au-delà du résultat technique, ce projet a constitué une expérience de formation riche. Il nous a confrontés à des problématiques réelles : l'alignement des étiquettes d'un modèle pré-entraîné, la gestion des classes peu fiables, l'optimisation de l'inférence sur mobile et la conception d'une interface ergonomique. Chaque difficulté a été l'occasion d'approfondir nos compétences en développement mobile, en apprentissage profond et en ingénierie logicielle.

Plusieurs **perspectives d'évolution** se dégagent. À court terme, l'application pourrait être enrichie d'une fonctionnalité de génération de rapports PDF exportables et de la prise en charge du format DICOM. À moyen terme, une synchronisation optionnelle et chiffrée avec un serveur permettrait le partage sécurisé des dossiers entre praticiens. Enfin, le ré-entraînement des modèles sur des données locales, idéalement validées par des radiologues marocains, améliorerait la fiabilité des prédictions, en particulier pour les classes actuellement exclues du diagnostic principal.

Nous tenons enfin à rappeler que PulmoScan AI est conçu comme un **outil d'aide à la décision** et ne saurait en aucun cas se substituer au jugement clinique d'un

professionnel de santé qualifié.

Bibliographie

- [1] G. Huang, Z. Liu, L. van der Maaten et K. Q. Weinberger, *Densely Connected Convolutional Networks (DenseNet)*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
- [2] O. Ronneberger, P. Fischer et T. Brox, *U-Net : Convolutional Networks for Biomedical Image Segmentation*, Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015.
- [3] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri et R. M. Summers, *ChestX-ray8 : Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
- [4] J. P. Cohen et al., *TorchXRayVision : A library of chest X-ray datasets and models*, 2022.
- [5] Google, *Flutter Documentation*, <https://docs.flutter.dev>. Consulté en 2026.
- [6] Google, *TensorFlow Lite Documentation*, <https://www.tensorflow.org/lite>. Consulté en 2026.

ANNEXE A

Annexes

A.1 Annexe A – Liste complète des 14 pathologies

L'ordre ci-dessous correspond aux indices 0 à 13 de la sortie du modèle DenseNet121 (TorchXRayVision).

TABLE A.1 – Les 14 pathologies classifiées et leur fiabilité

Indice	Pathologie	Remarque
0	Atelectasis	Fiable
1	Consolidation	Exclue (AUC < 0,80)
2	Infiltration	Exclue (AUC < 0,80)
3	Pneumothorax	Fiable
4	Edema	Fiable
5	Emphysema	Fiable
6	Fibrosis	Fiable
7	Effusion	Fiable
8	Pneumonia	Exclue (AUC < 0,80)
9	Pleural_Thickening	Fiable
10	Cardiomegaly	Fiable
11	Nodule	Exclue (AUC < 0,80)
12	Mass	Fiable
13	Hernia	Fiable

Indices non entraînés

Les indices 14 à 17 de la sortie du modèle ne sont pas entraînés et produisent une valeur constante d'environ 0,5. Ils sont systématiquement ignorés lors du

post-traitement.

A.2 Annexe B – Extrait du chargement des modèles

```
1 class AIService {
2   static final AIService _instance = AIService._internal();
3   factory AIService() => _instance;
4   AIService._internal();
5
6   Interpreter? _classifier;
7   Interpreter? _segmenter;
8
9   Future<void> loadModels() async {
10    try {
11      _classifier = await Interpreter.fromAsset(
12        'assets/models/pulmoscan_model.tflite');
13      _segmenter = await Interpreter.fromAsset(
14        'assets/models/unet_lung.tflite');
15    } catch (e) {
16      // Repli : mode simulation si les modeles sont absents
17      _classifier = null;
18      _segmenter = null;
19    }
20  }
21 }
```

Listing A.1 – Chargement des modèles TFLite avec mécanisme de repli (Dart, simplifié)

A.3 Annexe C – Maquettes Figma

[**FIGURE**]

Planche de maquettes Figma présentant les premières versions des écrans de l'application (Dashboard, Patients, Nouvel examen, Résultats).